Databases for operators

Run relational databases with confidence: SQL, indexes and plans, transactions, backups, replication, safe migrations, caches and incident triage.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Relational databases and SQL for operators (15 min)
2. Indexes and reading query plans (15 min)
3. Transactions, isolation and locking (15 min)
4. Backups, point-in-time recovery and restore testing (15 min)
5. Replication, high availability and failover (16 min)
6. Database performance, memory and connection pooling (15 min)
7. Safe schema migrations and zero-downtime changes (15 min)
8. NoSQL, caches and choosing the right data store (15 min)
9. Postgres vacuum, bloat and routine maintenance (13 min)
10. Schema design that is easy to operate (12 min)
11. Debugging database problems under pressure (16 min)
12. Database observability and capacity planning (12 min)
Cheat sheet

Lesson 1. Relational databases and SQL for operators

15 minutes. Understand tables, keys and relationships, and read and write the SQL that operators use to inspect data, check sizes and find trouble.

Almost every service you will run keeps its important data in a database. You do not need to design schemas all day, but you must read a query, understand what a table holds and know which commands are safe on a live system.

What you will learn

- Tables, rows, primary keys and foreign keys
- SELECT, JOIN, GROUP BY and filtering
- Safe habits for UPDATE and DELETE
- Operator queries: sizes, counts and running sessions

The relational model

A **table** holds rows with the same columns. A **primary key** identifies each row. A **foreign key** points at a row in another table and lets the database enforce that the link is valid.

```
CREATE TABLE customers (
  id    bigserial PRIMARY KEY,
  email text NOT NULL UNIQUE,
  name  text NOT NULL
);

CREATE TABLE orders (
  id          bigserial PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES customers (id),
  total_cents integer NOT NULL CHECK (total_cents >= 0),
  created_at  timestamptz NOT NULL DEFAULT now()
);
```

Constraints (`NOT NULL`, `UNIQUE`, `CHECK`, `REFERENCES`) put the rules in the database, where every client has to obey them, not in each application.

Reading data

```
SELECT c.name, count(*) AS orders, sum(o.total_cents) / 100.0 AS revenue
FROM customers c
JOIN orders o ON o.customer_id = c.id
WHERE o.created_at >= now() - interval '30 days'
GROUP BY c.name
HAVING count(*) > 5
ORDER BY revenue DESC
LIMIT 10;
```

- `WHERE` filters rows before grouping, `HAVING` filters groups after
- `JOIN` combines rows from two tables on a condition
- `LIMIT` protects you from pulling a million rows into your terminal

Changing data safely

```
BEGIN;
SELECT count(*) FROM orders WHERE created_at < '2020-01-01';
DELETE FROM orders WHERE created_at < '2020-01-01';
-- check the row count reported matches your expectation, then:
COMMIT;   -- or ROLLBACK;
```

Habits that prevent disasters:

- Run the matching `SELECT` first and compare the count

MARGA by EknathaLabs marga.eknathalabs.com

- Use a transaction so you can `ROLLBACK`
- Never run `UPDATE` or `DELETE` without a `WHERE` clause. Many clients can be set to refuse this.
- Do large changes in small batches so locks and logs stay small
- Take a backup or snapshot before a risky change

Operator queries (PostgreSQL)

```
SELECT pg_size_pretty(pg_database_size(current_database()));

SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 10;

SELECT pid, state, now() - query_start AS running_for, left(query, 60) AS query
FROM pg_stat_activity
WHERE state <> 'idle'
ORDER BY running_for DESC;
```

These answer three everyday questions: how big is it, what is taking space, and what is running now. Other engines have equivalents, such as `information_schema` tables and `SHOW PROCESSLIST` in MySQL.

Using the command line

```
psql "host=db1.example.com dbname=shop user=readonly" -c "SELECT count(*) FROM orders;"
psql -h db1.example.com -U readonly shop -f report.sql
```

Use a read-only account for investigation. Inside `psql`, `\dt` lists tables, `\d orders` describes one, and `\x` switches to a readable vertical layout.

Diagram: Where a query's data comes from

Part	What it does	What goes wrong
Application or psql	Sends SQL over a network connection.	An over-privileged account can change or delete anything.
Parse and plan	The database checks syntax and chooses how to run it.	A bad plan makes a simple query slow.
Tables and indexes	Rows are read from memory or disk.	Cache misses turn queries into disk reads.
Write-ahead log	Changes are logged before they are acknowledged.	A full log disk stops all writes.

Flow: Application or psql to Parse and plan / Parse and plan to Tables and indexes / Tables and indexes to Write-ahead log

Incident: the delete without a WHERE

> **Incident:** *An engineer ran a cleanup query in a production console. The WHERE clause was cut off when pasting, and every row in a customer table was deleted.*

```
$ psql shop -c "DELETE FROM customers"
DELETE 1840223
```

The team restored from a point-in-time backup, losing twelve minutes of writes to reconcile by hand. Afterwards they required read-only consoles by default, a break-glass role for writes, transactions with a row-count check, and reviews for production data fixes.

Common mistakes

- Investigating with a read-write account
- Running unbounded queries on a busy primary
- Forgetting that DDL such as `DROP TABLE` may not roll back in every engine

MARGA by EknathaLabs
marga.eknathalabs.com

- Selecting * on wide tables during an incident

Check yourself

What does a primary key do?

Answer: Uniquely identifies each row

What is the difference between WHERE and HAVING?

Answer: WHERE filters rows before grouping, HAVING filters groups after aggregation

Why run the matching SELECT before a DELETE?

Answer: To confirm how many rows will be affected before changing anything

Which account should you use for investigation by default?

Answer: A read-only role

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Indexes and reading query plans

15 minutes. Know what an index does and costs, read EXPLAIN output to find sequential scans and bad estimates, and add the right index safely on a live system.

A table with ten rows is fast however you query it. A table with fifty million rows is only fast when the database can jump straight to the rows it needs. Indexes make that possible, and query plans show whether the database is using them.

What you will learn

- What an index is and when it helps
- The cost of indexes
- Reading `EXPLAIN` and `EXPLAIN ANALYZE`
- Why the planner sometimes ignores an index
- Creating indexes without blocking writes

What an index is

An index is a separate, sorted structure (usually a B-tree) that maps column values to row locations, like the index at the back of a book. Looking up a value needs a few page reads instead of reading the whole table.

```
CREATE INDEX idx_orders_customer ON orders (customer_id);
CREATE INDEX idx_orders_recent ON orders (created_at DESC);
CREATE INDEX idx_orders_open ON orders (customer_id) WHERE status = 'open';
CREATE UNIQUE INDEX idx_customers_email ON customers (lower(email));
```

- A **composite index** on `(customer_id, created_at)` helps queries that filter on `customer_id`, or on both. It does not help a query that filters only on `created_at`, because order matters.
- A **partial index** (with `WHERE`) is small and fast for a common subset.
- A **unique index** also enforces uniqueness.

The cost

Every index must be updated on every write, takes disk space and memory, and slows `INSERT`, `UPDATE` and `DELETE`. Add the indexes your queries need, and remove the ones nothing uses.

```
SELECT relname, indexrelname, idx_scan
FROM pg_stat_user_indexes
ORDER BY idx_scan ASC
LIMIT 10;
```

An index with `idx_scan` of zero for a long time is a candidate for removal (check replicas and rare reports first).

Reading a plan

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 42 ORDER BY created_at DESC LIMIT 20;

Limit  (cost=0.43..8.10 rows=20) (actual time=0.05..0.09 rows=20 loops=1)
  ->  Index Scan using idx_orders_customer on orders
        (cost=0.43..120.5 rows=310) (actual time=0.04..0.08 rows=20 loops=1)
        Index Cond: (customer_id = 42)
Planning Time: 0.2 ms
Execution Time: 0.12 ms
```

Look for these in order:

1. **Scan type**: `Seq Scan` on a big table is a warning. `Index Scan` or `Index Only Scan` is usually good.
2. **Estimated rows versus actual rows**. A large gap means the statistics are stale. Run `ANALYZE`.

MARGA by EknathaLabs marga.eknathalabs.com

3 **Where the time goes**: find the node with the largest actual time.

4 **Sorts that spill to disk** and nested loops over huge row counts.

`EXPLAIN` alone shows the plan without running the query. `EXPLAIN ANALYZE` actually runs it, so wrap writes in a transaction you roll back.

Why an index is not used

- The query returns a large share of the table, so a sequential scan is genuinely cheaper
- A function or cast on the column (`WHERE lower(email) = ...` needs an index on `lower(email)`)
- A leading wildcard (`LIKE '%abc'`)
- Mismatched types, such as comparing text to a number
- Stale statistics

Adding an index on a live system

A plain `CREATE INDEX` blocks writes to the table while it builds. On a busy table use:

```
CREATE INDEX CONCURRENTLY idx_orders_customer ON orders (customer_id);
```

It takes longer and cannot run inside a transaction, and a failed build leaves an invalid index that you must drop and retry. MySQL uses online DDL with `ALGORITHM=INPLACE` for similar results.

Diagram: Finding why a query is slow

Part	What it does	What goes wrong
Slow query found	From slow query log or monitoring.	Tuning random queries wastes time. Start with the worst by total time.
EXPLAIN ANALYZE	Shows scan type, row estimates and time per step.	Plain EXPLAIN hides the real timings.
Find the cause	Missing index, stale statistics, bad predicate or too many rows.	Adding an index without reading the plan may not help.
Fix and measure	Add an index concurrently or rewrite the query, then compare.	No before and after number means no proof.

Flow: Slow query found to EXPLAIN ANALYZE / EXPLAIN ANALYZE to Find the cause / Find the cause to Fix and measure

Incident: the index build that blocked checkout

> **Incident:** *At peak traffic, a plain CREATE INDEX on the orders table blocked every insert for eleven minutes. Checkout timed out for customers.*

```
$ psql shop -c "SELECT pid, wait_event_type, left(query,50) FROM pg_stat_activity WHERE wait_event \
    _type = 'Lock' LIMIT 3"
```

The team cancelled the build, re-ran it with `CREATE INDEX CONCURRENTLY` in a quiet period, and added index creation to the migration checklist.

Common mistakes

- Indexing every column
- Reading only plain EXPLAIN
- Wrong column order in composite indexes
- Building indexes without CONCURRENTLY at peak time

Check yourself

What is a cost of adding an index?

Answer: Writes become slower and storage grows

MARGA by EknathaLabs
marga.eknathalabs.com

What does a large gap between estimated and actual rows suggest?

Answer: Stale or poor statistics, so run ANALYZE and check the predicate

Which statement builds an index without blocking writes in PostgreSQL?

Answer: CREATE INDEX CONCURRENTLY

Why might the database use a sequential scan despite an index?

Answer: The query returns a large part of the table, so reading it all is cheaper

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Transactions, isolation and locking

15 minutes. Learn ACID, isolation levels, how locks and long transactions cause stalls, and how to find and resolve blocking and deadlocks.

A bank transfer must either move the money out and in, or do neither. Transactions give databases that promise. They also explain many production stalls, because a transaction that stays open too long holds locks that everyone else waits for.

What you will learn

- ACID in plain terms
- Isolation levels and the anomalies they prevent
- Row locks, table locks and deadlocks
- Finding blocking sessions and long transactions

ACID

Letter	Meaning	What it protects you from
Atomicity	All statements in a transaction apply, or none	Half-done transfers
Consistency	Constraints hold before and after	Orphaned or invalid rows
Isolation	Concurrent transactions do not see each other's partial work	Reading unfinished data
Durability	Committed data survives a crash	Losing acknowledged writes

Durability comes from the **write-ahead log (WAL)**: changes are written to the log and flushed to disk before the commit is acknowledged.

A transaction

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

If the server crashes between the two updates, neither remains after recovery.

Isolation levels

Level	Prevents	Notes
Read committed	Reading uncommitted data	Default in PostgreSQL. Each statement sees data committed before it began.
Repeatable read	Rows changing between two reads in one transaction	Default in MySQL InnoDB.
Serializable	All anomalies, as if transactions ran one after another	Can abort transactions that must be retried.

Stronger isolation costs more coordination. Applications using serializable mode must be ready to retry on a serialization failure.

Locks

Writers lock the rows they change until commit or rollback. Readers normally do not block writers (PostgreSQL uses MVCC, keeping old versions visible to readers).

```
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
```

MARGA by EknathaLabs marga.eknathalabs.com

`FOR UPDATE` locks selected rows so you can safely read then update them. Schema changes (`ALTER TABLE`) take stronger table-level locks, and a waiting `ALTER` can queue behind a long transaction while blocking everyone behind it.

Deadlock: transaction A holds row 1 and waits for row 2, while B holds row 2 and waits for row 1. The database detects it, aborts one transaction and reports an error. Prevent deadlocks by updating rows in a consistent order and keeping transactions short.

Find blockers

```
SELECT blocked.pid AS blocked_pid, blocking.pid AS blocking_pid,
       left(blocking.query, 60) AS blocking_query,
       now() - blocking.xact_start AS blocking_for
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
  ON blocking.pid = ANY (pg_blocking_pids(blocked.pid));

SELECT pid, state, now() - xact_start AS xact_age, left(query, 60)
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_age DESC LIMIT 5;
```

Look for sessions in state `idle in transaction`. They hold locks (and stop cleanup of old row versions) while doing nothing. Protect yourself with timeouts:

```
ALTER ROLE app SET idle_in_transaction_session_timeout = '30s';
ALTER ROLE app SET lock_timeout = '5s';
ALTER ROLE app SET statement_timeout = '30s';
```

To end a stuck session: `SELECT pg_cancel_backend(pid);` cancels the query, and `pg_terminate_backend(pid)` ends the connection. Fix the cause in the application afterwards.

Diagram: How one long transaction becomes an outage

Part	What it does	What goes wrong
Long open transaction	Holds row locks and an old snapshot.	Idle in transaction sessions are invisible without monitoring.
Schema change waits	ALTER TABLE queues for a lock behind the transaction.	The queued ALTER now blocks every new query.
Queries pile up	Connections fill with waiting statements.	The pool is exhausted and the app times out.
User-facing outage	Requests fail across the service.	Killing the wrong session may make it worse.

Flow: Long open transaction to Schema change waits / Schema change waits to Queries pile up / Queries pile up to User-facing outage

Incident: the migration that waited for a lock

> **Incident:** *A routine ALTER TABLE queued behind a report transaction that had been open for forty minutes. New queries queued behind the ALTER, and the site went down in seconds.*

```
$ psql shop -c "SELECT pid, state, now()-xact_start AS age FROM pg_stat_activity WHERE xact_start  \
    IS NOT NULL ORDER BY age DESC LIMIT 2"
```

They cancelled the ALTER first, which freed the queue, set `lock_timeout` on migrations so they fail fast, and moved reports to a replica.

Common mistakes

- Holding a transaction open while calling another service
- Running schema changes with no lock timeout
- Ignoring `idle in transaction` sessions
- Retrying deadlocks forever without finding the order problem

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What does atomicity guarantee?

Answer: All statements in a transaction apply or none do

What is a deadlock?

Answer: Two transactions each wait for a lock the other holds

Why are idle in transaction sessions harmful?

Answer: They hold locks and old snapshots while doing nothing

Why set lock_timeout on schema migrations?

Answer: So the migration fails quickly instead of queueing and blocking all other queries

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Backups, point-in-time recovery and restore testing

15 minutes. Choose between logical and physical backups, use WAL archiving for point-in-time recovery, set RPO and RTO targets, and prove restores work.

A backup you have never restored is a hope, not a backup. Databases are the part of your system that cannot be rebuilt from code, so backup design and restore practice are core operator skills.

What you will learn

- Logical and physical backups
- Point-in-time recovery (PITR)
- RPO and RTO as targets
- Storing backups safely
- Restore drills

Two kinds of backup

Kind	Example	Good for	Weak at
Logical	`pg_dump`, `mysqldump`	Small databases, moving between versions, single tables	Slow on large data, restores rebuild indexes
Physical	`pg_basebackup`, volume snapshots	Fast restore of large databases, replicas	Same major version and platform

```
pg_dump --format=custom --file=shop.dump shop
pg_restore --dbname=shop_restored --jobs=4 shop.dump
pg_basebackup -h db1.example.com -D /backups/base -X stream -P
```

Snapshots of a disk are only safe if the database is consistent at that moment. Use the database's backup mode or a snapshot method that is crash-consistent and combine with the log archive.

Point-in-time recovery

A base backup plus the continuous archive of the write-ahead log lets you restore to any moment, for example "one minute before the bad delete".

```
# postgresql.conf on the primary
wal_level = replica
archive_mode = on
archive_command = 'test ! -f /archive/%f && cp %p /archive/%f'

# recovery settings when restoring
restore_command = 'cp /archive/%f %p'
recovery_target_time = '2026-03-04 09:41:00+00'
recovery_target_action = 'promote'
```

Real systems copy the archive to remote object storage with a tool such as pgBackRest or WAL-G rather than a plain `cp`. Managed services (RDS, Cloud SQL, Azure Database) offer PITR as a feature, but you still decide retention and test the restore.

Targets: RPO and RTO

- **RPO** (recovery point objective): how much data you can afford to lose. With PITR and continuous archiving it is seconds to minutes. With nightly dumps it is up to 24 hours.
- **RTO** (recovery time objective): how long recovery may take. Restoring 2 TB takes hours, so test it.

Agree these numbers with the business first. They decide cost.

Store them safely

MARGA by EknathaLabs marga.eknathalabs.com

- Keep copies in a different account or region than the primary
- Encrypt backups and protect the keys separately
- Make a copy immutable or write-once, so ransomware or a mistaken delete cannot erase it
- Monitor that backups finished and that the archive is not falling behind
- Keep retention rules, and delete data you must delete under privacy rules

Prove it works

```
createdb shop_check
pg_restore --dbname=shop_check --exit-on-error shop.dump
psql shop_check -c "SELECT count(*) FROM orders;"
```

A restore drill checks that the files are complete, the keys work, the procedure is written correctly and the time matches the RTO. Run it on a schedule and automate it where possible: restore to a scratch instance, run a few sanity queries and compare row counts, then discard it.

Diagram: Point-in-time recovery

Part	What it does	What goes wrong
Base backup	A full physical copy taken at a known time.	An untested or corrupt base makes the archive useless.
WAL archive	A continuous stream of changes copied off the server.	A gap in the archive stops recovery at that point.
Replay to target	Restore the base, then apply logs up to the chosen time.	Picking the wrong target time restores the bad change too.
Verify and promote	Check data, then open for writes or extract the needed rows.	Promoting before checking can cement a mistake.

Flow: Base backup to Replay to target / WAL archive to Replay to target / Replay to target to Verify and promote

Incident: the backups that were empty

> **Incident:** *A company lost its production database to a mistaken deletion and found that none of its backup methods produced a usable copy: one failed silently for months, one had the wrong version and one was never configured.*

```
$ ls -la /backups/nightly | tail -3
-rw-r--r-- 1 postgres postgres 0 Jan 12 02:00 shop.dump
-rw-r--r-- 1 postgres postgres 0 Jan 13 02:00 shop.dump
```

The company recovered data from a staging copy that was six hours old. Many teams have been in a similar position, and the lesson is the same: monitor backup size and age, and restore regularly.

Common mistakes

- Backups on the same disk or account as the database
- Never testing a restore
- Alerts only on job exit codes, not on backup size or freshness
- Assuming a replica is a backup (it replicates deletions too)

Check yourself

Why is a replica not a backup?

Answer: It copies mistakes such as a dropped table almost immediately

What does point-in-time recovery require?

Answer: A base backup plus a continuous archive of write-ahead logs

What does RPO measure?

MARGA by Eknatha Labs marga.eknathalabs.com

Answer: How much data loss is acceptable, measured in time

What proves a backup is usable?

Answer: A regular restore test with sanity checks on the data

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Replication, high availability and failover

16 minutes. Understand primary and replica setups, synchronous and asynchronous replication, replication lag, failover and split brain, and read scaling.

One database server is a single point of failure. Replication keeps copies on other machines so you can survive a failure, spread read load, and do maintenance with little downtime. It also introduces new problems: lag, failover and the risk of two servers both believing they are in charge.

What you will learn

- Primary and replica roles
- Asynchronous and synchronous replication
- Replication lag and stale reads
- Failover and split brain
- Managed high availability

Primary and replicas

The **primary** accepts writes. **Replicas** receive the stream of changes (the write-ahead log in PostgreSQL, the binary log in MySQL) and apply them. Replicas can serve read-only queries and can be promoted to primary.

Asynchronous or synchronous

Mode	Commit returns when	Trade-off
Asynchronous	The primary has written locally	Fast, but a primary crash can lose the last few commits
Synchronous	At least one replica has confirmed	No acknowledged commit lost, but slower and a stuck replica can stall writes

Many teams use synchronous replication to a nearby replica for durability and asynchronous replicas elsewhere for reads and disaster recovery. Choose based on the RPO you agreed.

Lag

Replicas apply changes after the primary, so reads from a replica may be slightly old. That is **replication lag**.

```
SELECT now() - pg_last_xact_replay_timestamp() AS replay_delay;
SELECT client_addr, state, sent_lsn, replay_lsn,
       pg_wal_lsn_diff(sent_lsn, replay_lsn) AS bytes_behind
FROM pg_stat_replication;
```

Causes of lag: heavy writes, a slow replica disk, long queries on the replica that conflict with applying changes, or network limits. Effects include a user saving a profile and not seeing it on the next page. Solutions include reading from the primary right after a write, session stickiness, or waiting for the replica to reach a known position.

Failover

When the primary fails, a replica must be promoted and clients must follow.

```
pg_ctl promote -D /var/lib/postgresql/16/main
```

Doing it by hand is slow and error prone, so teams use an automated manager such as Patroni, or a managed service. A safe failover needs:

MARGA by EknathaLabs marga.eknathalabs.com

1. A reliable decision that the primary is really down (a majority of a consensus group such as etcd, not one health check)
2. **Fencing**: making sure the old primary cannot accept writes again
3. Promoting the most up-to-date replica
4. Pointing clients at the new primary through DNS, a virtual IP or a proxy
5. Rebuilding the old primary as a replica

Split brain happens when two servers both accept writes, causing divergent data that is hard or impossible to merge. Fencing and quorum exist to prevent it.

Managed services

Cloud databases offer multi-zone standby instances with automatic failover, typically within a minute or two, and read replicas as a separate feature. You still need to test failover, because connection strings, pools and application retry logic decide how long users notice an outage.

Diagram: A failover

Part	What it does	What goes wrong
Primary fails	Hardware, network or process failure.	A false alarm may trigger an unneeded failover.
Decide and fence	A quorum agrees, and the old primary is cut off.	Without fencing, two primaries can accept writes.
Promote a replica	The most up-to-date replica becomes the primary.	A lagging replica loses recent commits.
Redirect clients	DNS, a virtual IP or proxy moves traffic.	Cached DNS and old connections keep going to the failed node.

Flow: Primary fails to Decide and fence / Decide and fence to Promote a replica / Promote a replica to Redirect clients

Incident: the failover that made it worse

> **Incident:** *A short network partition made an automated tool promote a replica while the old primary was still running. For several minutes both accepted writes, and the team had to reconcile the data by hand.*

```
$ patronictl list
| Member | Role   | State   | Lag in MB |
| db1    | Leader | running |           |
| db2    | Leader | running |           |
```

A well-known public example is the October 2018 incident at a large code-hosting company, which wrote up a failover after a short network partition. In general, the repair was slow, data on both sides had to be compared, and the lasting fixes were better quorum rules, fencing and tested failover procedures.

Common mistakes

- Treating a replica as a backup
- Failing over without fencing the old primary
- Never rehearsing failover
- Reading from replicas for data that must be current

Check yourself

What is replication lag?

Answer: The delay before a replica applies changes made on the primary

What is split brain?

Answer: Two servers both accepting writes as primary

MARGA by EknathaLabs marga.eknathalabs.com

What does synchronous replication add?

Answer: Commits are acknowledged only after a replica confirms, so none are lost on failover

What does fencing do?

Answer: Ensures the old primary cannot accept writes after failover

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Database performance, memory and connection pooling

15 minutes. Find slow queries with statistics, understand memory and cache behaviour, size connections with a pooler, and spot bloat and vacuum problems.

When a service slows down, the database is often involved, but guessing wastes hours. A few views and settings tell you whether the problem is bad queries, too many connections, memory pressure or maintenance falling behind.

What you will learn

- Finding the most expensive queries
- Cache hit ratio and memory settings
- Why connections are expensive, and what a pooler does
- Vacuum, bloat and statistics

Find the worst queries

Enable the `pg_stat_statements` extension. It records time and calls per normalised query.

```
SELECT calls, round(total_exec_time::numeric, 0) AS total_ms,
       round(mean_exec_time::numeric, 2) AS mean_ms, left(query, 70) AS query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

Sort by **total** time first. A query that takes 5 ms but runs 10 million times costs more than one that takes 2 seconds once a day. MySQL has the slow query log and the Performance Schema for the same job. Take the worst few to `EXPLAIN ANALYZE`.

Memory and cache

Databases are fast when working data fits in memory.

```
SELECT round(100.0 * sum(blks_hit) / nullif(sum(blks_hit) + sum(blks_read), 0), 2) AS cache_hit_pc \
    t
FROM pg_stat_database;
```

A persistently low hit ratio on an OLTP database suggests too little memory or queries that scan too much. Key PostgreSQL settings:

- `shared_buffers`: the database's own cache, commonly around a quarter of RAM
- `work_mem`: memory per sort or hash **per operation**, so a large value times many sessions can exhaust RAM
- `effective_cache_size`: a hint about how much memory the OS cache provides

Treat default settings as a starting point and change one thing at a time with measurements.

Connections are expensive

Each PostgreSQL connection is a separate process that uses memory. Hundreds of idle connections from many application instances waste resources and make the server slow. A **connection pooler** such as PgBouncer sits in front and shares a small number of real connections among many clients.

```
[pgbouncer]
pool_mode = transaction
max_client_conn = 2000
default_pool_size = 20
```

- `transaction` mode returns a server connection to the pool after each transaction, which gives the best sharing but breaks features that rely on session state

MARGA by EknathaLabs
marga.eknathalabs.com

- Size the pool near the number of CPU cores times two or so, not the number of clients
- Check also the connection pool in your application framework. Many pools times many instances can still overload the database.

```
SELECT state, count(*) FROM pg_stat_activity GROUP BY state;
SHOW max_connections;
```

Vacuum and bloat

PostgreSQL does not overwrite rows. Updates and deletes leave old row versions that **autovacuum** removes later. If vacuum falls behind, tables and indexes grow (**bloat**) and queries slow. Long transactions prevent cleanup, which is another reason to keep them short.

```
SELECT relname, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 5;
```

Vacuum also freezes old transaction IDs. If that is blocked for too long, the database can stop accepting writes to avoid wraparound, a failure that has hit well-known services.

Quick triage order

1. CPU, memory, disk latency and disk space on the server
2. Connection count and waiting sessions
3. Top queries by total time
4. Locks and long transactions
5. Replication lag and vacuum progress

Diagram: Slow database triage

Part	What it does	What goes wrong
Host resources	CPU, memory, disk latency, free space.	A full or slow disk makes every query slow.
Connections	Count, state and waiting sessions.	Exhausted connections cause timeouts with low CPU.
Top queries	pg_stat_statements sorted by total time.	Fixing a rare slow query ignores the frequent cheap one.
Locks and vacuum	Long transactions, blocked sessions, dead rows.	Bloat grows quietly until it hurts.
Change one thing	Add an index, pool, tune or rewrite, then measure.	Several changes at once hide what worked.

Flow: Host resources to Connections / Host resources to Top queries / Connections to Locks and vacuum / Top queries to Locks and vacuum / Locks and vacuum to Change one thing

Incident: the connection storm

Incident: *After a deploy, forty new application instances each opened a pool of fifty connections. The database ran out of memory and every service using it slowed down.*

```
$ psql -c "SELECT count(*) FROM pg_stat_activity"
 count
-------
  1987
```

The team reduced each pool to five connections, placed PgBouncer in front and added an alert on connection use. They also made the deploy tool compute pool sizes from a shared connection budget.

Common mistakes

- Raising max_connections instead of pooling
- Setting work_mem huge for all sessions

MARGA by EknathaLabs marga.eknathalabs.com

- Looking only at the slowest single query
- Ignoring autovacuum warnings

Check yourself

Why sort pg_stat_statements by total time first?

Answer: Frequent cheap queries can cost more overall than one slow query

What does a connection pooler do?

Answer: Shares a small number of database connections among many clients

Why can a large work_mem be dangerous?

Answer: It applies per operation per session, so many sessions can exhaust RAM

What does bloat mean?

Answer: Tables and indexes holding dead row versions because vacuum has not kept up

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Safe schema migrations and zero-downtime changes

15 minutes. Change a live schema without outages using the expand and contract pattern, online index and column changes, batched backfills and migration tooling.

Application code can be rolled back in seconds. A schema change on a table with a hundred million rows may lock it for minutes and cannot be undone by redeploying. Safe migrations are a skill of their own.

What you will learn

- Migration tools and versioned changes
- Which operations take dangerous locks
- The expand and contract pattern
- Backfilling data in batches
- Checks before running a migration in production

Versioned migrations

Keep every schema change as a numbered file in version control, applied in order by a tool (Flyway, Liquibase, Alembic, Rails or Django migrations, or Atlas). The tool records what has run in a table, so every environment ends in the same state and reviewers see the change before it ships.

```
-- 0042_add_orders_status.sql
ALTER TABLE orders ADD COLUMN status text;
```

Run migrations from the pipeline, not from a laptop, and run them before the code that needs them.

Operations that hurt

Operation	Risk on a large table	Safer approach
Add a column with a volatile default (older versions)	Rewrites the whole table	Add nullable, backfill in batches, then set the default
Add `NOT NULL`	Scans the table under a strong lock	Add a `CHECK ... NOT VALID` constraint, validate separately
Add a foreign key	Locks and validates both tables	Add `NOT VALID`, then `VALIDATE CONSTRAINT`
Create an index	Blocks writes	`CREATE INDEX CONCURRENTLY`
Change a column type	Rewrites the table	Add a new column and migrate
Rename a column	Breaks running code	Expand and contract

Always set a lock timeout, so a migration that cannot get its lock fails fast instead of queueing:

```
SET lock_timeout = '3s';
SET statement_timeout = '60s';
ALTER TABLE orders ADD COLUMN status text;
```

Retry in a quiet moment if it fails.

Expand and contract

To rename `customer_name` to `full_name` without downtime, do it in separate deploys:

1. **Expand**: add the new column `full_name` (nullable)
2. **Write both**: deploy code that writes the old and the new column
3. **Backfill**: copy existing data in batches
4. **Read the new column**: deploy code that reads `full_name`
5. **Stop writing the old column**

MARGA by EknathaLabs marga.eknathalabs.com

6 **Contract**: drop `customer_name` in a later release, after a safety period

Each step is backward compatible with the previous version of the code, so a rollback is always possible until the final drop.

Backfill in batches

```
UPDATE orders SET full_name = customer_name
WHERE id BETWEEN 1 AND 10000 AND full_name IS NULL;
```

```
for start in $(seq 1 10000 1000000); do
  psql shop -c "UPDATE orders SET full_name = customer_name WHERE id BETWEEN $start AND $((start + \
     9999)) AND full_name IS NULL"
  sleep 0.5
done
```

Small batches keep transactions short, avoid replication lag spikes, and can be stopped and resumed. Watch replica lag while it runs and slow down if it grows.

Before production

- Rehearse on a restored copy of production-sized data and record how long it takes
- Review the lock each statement takes
- Have a rollback or forward-fix plan
- Take a fresh backup or know your PITR window
- Do not combine a risky migration with a code release in the same deploy

Diagram: Expand and contract

Part	What it does	What goes wrong
Expand	Add the new column or table, nullable.	Adding it with a heavy default locks the table.
Write both	Code writes old and new, then data is backfilled.	Forgetting rows written before dual-write leaves gaps.
Read new	Switch reads to the new column.	Switching before the backfill finishes returns missing data.
Contract	Drop the old column after a safe period.	Dropping too early removes your rollback.

Flow: Expand to Write both / Write both to Read new / Read new to Contract

Incident: the column rename

Incident: *A single deploy renamed a column and shipped the code that used the new name. During the rolling update, old application instances still queried the old name and returned errors for seven minutes.*

```
$ kubectl logs deploy/shop | grep -c 'column "customer_name" does not exist'
18342
```

The team adopted expand and contract for all renames, added a rule that migrations must work with both the old and new code versions, and tested that rule in staging by running the previous release against the new schema.

Common mistakes

- Running a table rewrite at peak time
- Migration and code in one irreversible step
- No lock timeout
- Skipping a rehearsal on realistic data

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What is expand and contract?

Answer: Making schema changes in backward compatible steps across several deploys

Why batch a backfill?

Answer: To keep transactions short and limit locks and replication lag

Why set lock_timeout before ALTER TABLE?

Answer: So the statement fails quickly instead of queueing and blocking other queries

When should the old column be dropped?

Answer: Later, after all code uses the new column and a safety period has passed

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. NoSQL, caches and choosing the right data store

15 minutes. Compare relational, key-value, document, wide-column and search stores, understand caches such as Redis and their failure modes, and choose with trade-offs in mind.

Relational databases are the right default, but not the only tool. Operators meet caches, queues, document stores and managed key-value services, and each has distinct failure modes. Choosing well starts from the access pattern, not from fashion.

What you will learn

- The main families of data stores
- The CAP trade-off and eventual consistency in plain terms
- Caching patterns and cache failure modes
- Operating Redis safely
- A checklist for choosing

Families

Family	Examples	Good at	Watch out for
Relational	PostgreSQL, MySQL	Joins, constraints, transactions	Scaling writes beyond one node
Key-value	Redis, DynamoDB	Fast lookups by key, huge scale	Limited queries
Document	MongoDB	Flexible records, nested data	Consistency across documents
Wide-column	Cassandra	High write rates across regions	Query design must come first
Search	Elasticsearch, OpenSearch	Full text and filtering	Not a primary source of truth
Time series	Prometheus, TimescaleDB	Metrics over time	High cardinality cost

Consistency in plain terms

When data is spread over several machines, a network partition forces a choice: keep answering (**availability**) and risk stale answers, or refuse some requests to stay correct (**consistency**). This is the idea behind CAP. Many stores offer **eventual consistency**: replicas converge shortly after a write, but a read right away may be stale. Some let you pick per request, for instance strongly consistent reads at higher cost.

Caching

A cache keeps copies of expensive results in fast memory. Common patterns:

- **Cache-aside**: the application checks the cache, on a miss reads the database and writes the result back with an expiry (TTL)
- **Write-through**: writes go to the cache and the database together
- **Invalidate on change**: delete or update the cached item when the source changes

Failure modes to plan for:

- **Cold cache**: after a restart every request misses and hits the database at once
- **Stampede**: a popular key expires and thousands of requests rebuild it together. Add jitter to TTLs and let only one request rebuild.

MARGA by EknathaLabs marga.eknathalabs.com

- **Stale data**: choose TTLs deliberately and invalidate on writes
- **Cache as the only copy**: if you cannot rebuild it, it is a database and needs backups and persistence

Operating Redis safely

```
redis-cli INFO memory | grep -E "used_memory_human|maxmemory_human"
redis-cli INFO stats | grep -E "keyspace_hits|keyspace_misses|evicted_keys"
redis-cli --scan --pattern 'session:*' | head
redis-cli SLOWLOG GET 5

maxmemory 2gb
maxmemory-policy allkeys-lru
appendonly yes
```

- Set `maxmemory` and an eviction policy, or Redis will grow until the host runs out of memory
- Never run `KEYS *` on production, because it blocks the single-threaded server. Use `SCAN`.
- Persistence (`appendonly`, snapshots) is optional for a pure cache and important for queues or sessions
- Do not expose Redis to the internet. Use a private network, authentication and TLS.

How to choose

Ask in order:

1. What are the main queries and how often? Start from reads and writes.
2. Do you need transactions and joins across entities? Pick relational.
3. How much data and what write rate in a year? Does one node suffice?
4. What consistency can users tolerate?
5. Who will operate it, and do you have backups, monitoring and skills for it?
6. Is a managed service available?

A boring relational database plus a cache covers a very large share of real systems. Adding another store adds a new thing to back up, secure, monitor and patch.

Diagram: Cache-aside read path

Part	What it does	What goes wrong
Application	Asks the cache first.	A cache outage must not take the app down.
Cache	Returns a hit, or a miss on absent or expired keys.	Eviction and expiry cause sudden misses.
Database	Source of truth for a miss.	A mass miss can overload it.
Store with TTL	The result is written back with an expiry.	A long TTL serves stale data.

Flow: Application to Cache / Application to Database / Database to Store with TTL / Store with TTL to Cache

Incident: the restart that melted the database

> **Incident:** *During maintenance a team restarted its Redis cache. Every request missed, the database received the full traffic it normally never saw, and it fell over within a minute.*

```
$ redis-cli INFO stats | grep keyspace
keyspace_hits:0
keyspace_misses:482113
```

They added gradual cache warming, rate-limited the rebuilds, replicas for the cache, and load tested the database with the cache turned off to learn its real limits.

Common mistakes

- Choosing a store because it is popular
- Treating a cache as durable storage
- Running KEYS on production
- No memory limit or eviction policy

Check yourself

What is cache-aside?

Answer: The application checks the cache, and on a miss loads from the database and fills the cache

What is a cache stampede?

Answer: Many requests rebuilding the same expired key at once

Why avoid KEYS * on production Redis?

Answer: It scans the whole keyspace in one blocking call that stalls other clients

What does eventual consistency mean?

Answer: Replicas converge after a short delay, so a read right after a write may be stale

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Postgres vacuum, bloat and routine maintenance

13 minutes. Understand why Postgres needs vacuum, how to spot bloat and long transactions, and which maintenance keeps the database healthy.

A table holds 2 million live rows and occupies 40 GB. Queries get slower every week. The cause is not the data; it is dead row versions that vacuum has not been able to clean. Postgres keeps old versions for concurrent readers, and maintenance decides whether that is harmless or costly.

What you will learn

- Why updates and deletes leave dead rows
- What vacuum and autovacuum do
- Finding bloat and blockers such as long transactions
- Routine maintenance and safe fixes

Dead rows and MVCC

Postgres never edits a row in place. An update writes a new version and marks the old one as dead once no transaction can still see it. Dead versions occupy space until **vacuum** marks it reusable. If dead space grows faster than it is cleaned, tables and indexes **bloat**: they use more disk, more cache and more time per scan.

Autovacuum

The autovacuum daemon vacuums and analyzes tables automatically when changes pass a threshold: by default 20 percent of the table plus 50 rows. On a huge, busy table, 20 percent is far too late, so tune it per table:

```
ALTER TABLE orders SET (
  autovacuum_vacuum_scale_factor = 0.02,
  autovacuum_analyze_scale_factor = 0.01
);
```

The same daemon protects against **transaction ID wraparound**, a rare but severe condition in which the database stops accepting writes. Monitor the age of the oldest transaction ID and alert well before the limit.

What blocks vacuum

Vacuum cannot remove a dead row that an old transaction might still see. Long-running transactions, forgotten sessions "idle in transaction", abandoned replication slots and stuck prepared transactions hold back cleanup.

```
SELECT pid, state, now() - xact_start AS age, left(query, 60) AS query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_start
LIMIT 5;

SELECT relname, n_live_tup, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 5;
```

Set `idle_in_transaction_session_timeout` so a forgotten session cannot block cleanup for days.

Routine care and fixes

- Plain `VACUUM` does not shrink the file; it makes space reusable. That is usually enough.
- `VACUUM FULL` rewrites the table and takes an exclusive lock, so use it rarely, in a window. Tools such as `pg_repack` rebuild with only a short lock.

MARGA by EknathaLabs marga.eknathalabs.com

- Rebuild bloated indexes with `REINDEX INDEX CONCURRENTLY`.
- Keep statistics fresh with `ANALYZE`; the planner depends on them.
- Watch dead tuple counts, table size growth, and the oldest transaction age as regular dashboard items.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Postgres vacuum, bloat and routine maintenance fits together

Part	What it does	What goes wrong
Dead rows and MVCC	Postgres never edits a row in place. An update writes a new version and marks the old one as dead once no transaction can still see it.	Using VACUUM FULL on a busy production table.
Autovacuum	The autovacuum daemon vacuums and analyzes tables automatically when changes pass a threshold: by default 20 percent of the table plus 50 rows.	Leaving autovacuum thresholds at the defaults for huge tables.
What blocks vacuum	Vacuum cannot remove a dead row that an old transaction might still see.	Ignoring idle-in-transaction sessions.
Routine care and fixes	Plain VACUUM does not shrink the file; it makes space reusable.	Not monitoring transaction ID age.

Flow: Dead rows and MVCC to Autovacuum / Autovacuum to What blocks vacuum / What blocks vacuum to Routine care and fixes

Incident: The table that kept growing

Incident: *A queue-like table with constant updates grew from 5 GB to 60 GB in a month, and dashboards slowed down.*

```
=> SELECT pid, state, now()-xact_start AS age FROM pg_stat_activity WHERE state='idle in transacti \
    on';
  pid  |        state        |       age
 4172  | idle in transaction  | 11 days 03:12:44
```

A reporting tool had left a transaction open for eleven days, so vacuum could not remove any row version created since. The session was ended, a timeout for idle transactions was added, and the table was rebuilt online with pg_repack. Per-table autovacuum settings and an alert on the oldest transaction age keep it from happening again.

Common mistakes

- Using VACUUM FULL on a busy production table
- Leaving autovacuum thresholds at the defaults for huge tables
- Ignoring idle-in-transaction sessions
- Not monitoring transaction ID age

Check yourself

Why does Postgres create dead rows?

Answer: Updates and deletes write new versions and leave old ones until vacuum cleans them

What can prevent vacuum from removing dead rows?

Answer: Long-running or idle-in-transaction sessions and stale replication slots

What does plain VACUUM do?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Marks dead space as reusable without shrinking the file

How can you avoid long-held idle transactions?

Answer: Set idle_in_transaction_session_timeout

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Schema design that is easy to operate

12 minutes. Choose keys, types, constraints and table layouts that keep a database fast, safe to change and easy to run.

The application team picked a text column for dates, no constraints, and one giant table for everything. It worked in the demo. A year later, bad data is everywhere, every migration locks the table, and nobody can say what a column means. Good design is partly an operations decision.

What you will learn

- Keys and types that age well
- Constraints as a safety net
- Layouts that ease growth: partitioning and archiving
- Designing for change: nullable columns, defaults and naming

Keys and types

Use a stable **primary key** that never changes and carries no business meaning: a bigint identity or a UUID. Use proper types: `timestamptz` for times (stored in UTC), `numeric` for money, `boolean`, `jsonb` for flexible parts. Dates stored as text cannot be compared, indexed or validated correctly. Pick sizes with growth in mind: an `integer` key overflows at about 2.1 billion rows, and changing it later means rewriting the table.

Constraints

Let the database refuse bad data:

```
CREATE TABLE payments (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id    bigint NOT NULL REFERENCES orders(id),
  amount      numeric(12,2) NOT NULL CHECK (amount > 0),
  currency    char(3) NOT NULL,
  status      text NOT NULL CHECK (status IN ('pending','paid','failed')),
  created_at  timestamptz NOT NULL DEFAULT now(),
  UNIQUE (order_id, created_at)
);
```

`NOT NULL`, `CHECK`, `UNIQUE` and foreign keys are cheap and catch bugs from every application, script and manual fix. Add them from the start; adding them later requires cleaning existing data first.

Layouts for growth

Tables that grow without limit (events, logs, payments) become slow to vacuum, back up and index. Use **partitioning** by time so old data sits in separate partitions that you can detach and archive quickly:

```
CREATE TABLE events (
  id bigint GENERATED ALWAYS AS IDENTITY,
  happened_at timestamptz NOT NULL,
  payload jsonb
) PARTITION BY RANGE (happened_at);

CREATE TABLE events_2026_01 PARTITION OF events
  FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');
```

Dropping a month is then an instant `DROP TABLE`, not a slow delete that leaves bloat. Automate partition creation so you never run out of them.

MARGA by EknathaLabs
marga.eknathalabs.com

Designing for change

Make future migrations safe. Add new columns as nullable or with a cheap default, backfill in batches, then add constraints. Avoid renaming columns used by running code without a compatibility period. Keep names clear and consistent (`created_at`, `customer_id`), and document the meaning of non-obvious columns with `COMMENT ON`. Keep rarely used wide data (big documents, blobs) out of hot tables. Review each schema change with a question: what happens when this table has a hundred times more rows?

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Schema design that is easy to operate fits together

Part	What it does	What goes wrong
Keys and types	Use a stable primary key that never changes and carries no business meaning: a bigint identity or a UUID.	Integer keys on tables that can grow large.
Constraints	Let the database refuse bad data.	Missing constraints, so bad data accumulates.
Layouts for growth	Tables that grow without limit (events, logs, payments) become slow to vacuum, back up and index.	One unbounded table with no archive plan.
Designing for change	Make future migrations safe. Add new columns as nullable or with a cheap default, backfill in batches, then add constraints.	Renaming or dropping columns without a compatibility period.

Flow: Keys and types to Constraints / Constraints to Layouts for growth / Layouts for growth to Designing for change

Incident: The table nobody could change

Incident: *A 900 million row events table needed a new column, and the migration locked writes long enough to cause an outage.*

```
=> ALTER TABLE events ALTER COLUMN user_id TYPE bigint;
-- waiting for ACCESS EXCLUSIVE lock ...
=> SELECT count(*) FROM events;
  count
-----------
 912330417
```

The team had used an integer key that was about to overflow, in a single unpartitioned table. The fix needed a phased plan: a new partitioned table, dual writes, batched copy, and a switch. Afterward the company adopted schema guidelines: bigint keys, timestamptz, constraints from the start, time partitioning for event tables, and a review checklist for every migration.

Common mistakes

- Integer keys on tables that can grow large
- Missing constraints, so bad data accumulates
- One unbounded table with no archive plan
- Renaming or dropping columns without a compatibility period

Check yourself

Which type is best for timestamps?

Answer: timestamptz, storing UTC

Why add constraints in the database?

Answer: They block bad data from every application, script and manual change

MARGA by EknathaLabs marga.eknathalabs.com

What is the benefit of time partitioning for event tables?

Answer: Old data can be detached or dropped quickly without bloat

How do you add a new column safely to a large table?

Answer: Add it nullable or with a cheap default, backfill in batches, then add constraints

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Debugging database problems under pressure

16 minutes. Triage slow, blocked, full or unreachable databases with a fixed order of checks, from connections and locks to disk, replication and recovery.

Database incidents are tense because the data matters and everything depends on it. A fixed order of checks keeps you calm and stops you from making an unrecoverable change in a hurry.

What you will learn

- A triage order for the four common symptoms
- Queries for connections, locks, disk and replication
- Safe emergency actions and their costs
- When to stop and protect the data

Principle: do no harm first

Before you kill, restart, failover or delete anything, ask: can this lose data, and is there a recent backup? Look before you act, take notes of what you ran, and prefer reversible steps (cancel a query before killing a connection, add capacity before restarting).

Symptom 1: it is slow

1. Host: CPU, memory, disk latency (`iostat -x 1`), swap
2. Sessions: how many active, how many waiting

```
SELECT wait_event_type, wait_event, count(*)
FROM pg_stat_activity WHERE state = 'active'
GROUP BY 1, 2 ORDER BY 3 DESC;
```

1. Top queries in the last interval (`pg_stat_statements`), and the plan of the worst
2. Recent changes: deploy, migration, traffic spike, stale statistics (`ANALYZE`)

Symptom 2: queries hang

Look for blocking: long transactions, a queued `ALTER`, an `idle in transaction` session.

```
SELECT pid, pg_blocking_pids(pid) AS blocked_by, state, left(query, 60)
FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

Cancel the **root** blocker, not the victims: `SELECT pg_cancel_backend(PID);` first, then `pg_terminate_backend(PID)` if needed.

Symptom 3: out of space

```
df -h /var/lib/postgresql
du -sh /var/lib/postgresql/16/main/* | sort -h | tail -5
```

```
SELECT slot_name, active, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS re \
    tained
FROM pg_replication_slots;
```

Common causes: write-ahead logs piling up because of a stuck replica or an unused replication slot, a failing archive command, runaway temp files, bloat, or a huge table that grew. Free space safely by fixing the cause (drop an abandoned slot, repair archiving), or adding disk. **Never delete files inside the data directory by hand**, because that can destroy the database.

Symptom 4: cannot connect

```
pg_isready -h db1.example.com -p 5432
ss -tlnp | grep 5432
journalctl -u postgresql --since "10 min ago"
```

MARGA by EknathaLabs marga.eknathalabs.com

Check in order: is the process up, is it listening, can the network reach it (firewall, security group, DNS), is it out of connections (`FATAL: too many clients`), does authentication fail (`pg_hba.conf`, password, TLS), did a failover move the primary?

Replication problems

```
SELECT client_addr, state, pg_wal_lsn_diff(sent_lsn, replay_lsn) AS bytes_behind
FROM pg_stat_replication;
```

A lagging replica may need a long query cancelled, more IO, or a rebuild from a fresh base backup.

Corruption and recovery

If you see checksum errors, missing files or crash loops, stop writing, **copy the data directory aside if you can**, and use your restore procedure rather than experimenting on the only copy. Document the timeline for the review.

Diagram: Database incident triage

Part	What it does	What goes wrong
Name the symptom	Slow, hanging, full or unreachable.	Mixing symptoms sends you down the wrong path.
Look without changing	Sessions, locks, disk, replication, logs.	Acting before looking can lose data.
Safest action first	Cancel a query, drop a stale slot, add capacity.	Restarts hide the evidence and drop the cache.
Confirm recovery	Latency, errors and replication back to normal.	Declaring it fixed too early invites a repeat.

Flow: Name the symptom to Look without changing / Look without changing to Safest action first / Safest action first to Confirm recovery

Incident: the full disk from a forgotten slot

> **Incident:** *A replica was decommissioned but its replication slot stayed. The primary kept every write-ahead log segment for it, filled the disk in the early hours and stopped accepting writes.*

```
$ psql -c "SELECT slot_name, active FROM pg_replication_slots"
 slot_name | active
-----------+--------
 replica_old | f
```

The team dropped the slot, the logs were recycled and writes resumed. They added alerts on disk free space, slot lag and WAL size, and a checklist item to remove slots when removing replicas.

Common mistakes

- Killing victim sessions instead of the root blocker
- Deleting files in the data directory
- Restarting before capturing evidence
- Running heavy diagnostic queries on an overloaded primary without limits

Check yourself

In a lock pile-up, which session do you cancel first?

Answer: The root blocker at the head of the chain

Why is deleting WAL files by hand dangerous?

Answer: The database may still need them for recovery or replication, and removing them can make it unrecoverable

What is the first question before a risky emergency action?

Answer: Could this lose data, and is there a recent usable backup?

MARGA by EknathaLabs marga.eknathalabs.com

Which tool checks if PostgreSQL accepts connections?

Answer: pg_isready

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Database observability and capacity planning

12 minutes. Choose the database metrics that matter, build useful dashboards and alerts, and forecast storage, connections and I/O before they run out.

Databases hold the state of everything else, so their failures are the most costly. A small, consistent set of metrics and alerts catches most problems before users do.

What you will learn

- The metrics that matter for a database
- Symptom alerts versus cause alerts
- Forecasting disk, connections and I/O
- Dashboards for on-call

What to collect

- **Latency and throughput:** queries per second and slow query rate.
- **Errors:** failed statements, deadlocks and timeouts.
- **Saturation:** connections used versus maximum, CPU, memory and disk I/O wait.
- **Locks and long transactions:** waiting sessions and the oldest transaction age.
- **Replication:** lag in seconds and bytes.
- **Storage:** data size, WAL or log size, free disk and growth rate.
- **Cache:** hit ratio, which shows if memory is too small.

```
# connection saturation
pg_stat_activity_count / pg_settings_max_connections > 0.8

# replication lag
pg_replication_lag_seconds > 30

# disk full in less than 7 days
predict_linear(node_filesystem_avail_bytes{mountpoint="/var/lib/postgresql"}[6h], 7*24*3600) < 0
```

Alert design

Alert on **symptoms** that matter to users, such as error rate or latency, and use cause metrics, such as lock waits or connection counts, for diagnosis and for early warnings that go to tickets. Page only when action is needed now. Add a link to the runbook in each alert.

Capacity

Record daily peaks and growth for storage, connections and I/O. Forecast with trend and seasonality, and compare the result with lead times for resizing, which on some platforms means a maintenance window. Review the numbers each month and before known peak events.

Dashboard

For on-call, build one page: traffic, latency, errors, connections, locks, replication, disk and the top queries by total time. Keep the layout the same for every database.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Database observability and capacity planning fits together

Part	What it does	What goes wrong
What to collect	Latency and throughput: queries per second and slow query rate.	Ignoring connection and lock metrics.
Alert design	Alert on symptoms that matter to users, such as error rate or latency, and use cause metrics, such as lock waits or connection counts, for diagnosis and for early...	Alerting only when something is already down.
Capacity	Record daily peaks and growth for storage, connections and I/O.	Capacity forecasts that ignore lead time.
Dashboard	For on-call, build one page: traffic, latency, errors, connections, locks, replication, disk and the top queries by total time.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: What to collect to Alert design / Alert design to Capacity / Capacity to Dashboard

Incident: Out of connections on the busiest day

Incident: *Connections reached the limit during peak traffic.*

```
FATAL: sorry, too many clients already
connections: 500 of 500
trend: +6% per month for 14 months
```

Each new service had added connections, and nobody watched the total. The team added a saturation alert at 80 percent, introduced a pooler, set per-service limits and reviewed the connection budget in capacity planning.

Common mistakes

- Alerting only when something is already down
- No baseline for normal behavior
- Ignoring connection and lock metrics
- Capacity forecasts that ignore lead time

Check yourself

Which metric shows connection saturation?

Answer: Connections used versus the maximum

What is a symptom alert?

Answer: Alerts on user-visible impact

What does low cache hit ratio suggest?

Answer: Memory may be too small

Why include lead time in forecasts?

Answer: Resizing may take time

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Databases (PostgreSQL)

SQL basics, operator queries, indexes and plans, locks, backups, replication, pooling and safe migrations.

Connect and explore

```
psql "host=db1.example.com dbname=shop user=readonly"
psql -h db1.example.com -U readonly shop -c "SELECT count(*) FROM orders;"
pg_isready -h db1.example.com -p 5432
```

psql command	Meaning
`\dt`	List tables
`\d orders`	Describe a table
`\x`	Toggle vertical output
`\timing`	Show query time

Safe changes

```
BEGIN;
SELECT count(*) FROM orders WHERE created_at < '2020-01-01';
DELETE FROM orders WHERE created_at < '2020-01-01';
COMMIT;   -- or ROLLBACK;
```

Never run `UPDATE` or `DELETE` without `WHERE`. Use a read-only role for investigation.

Operator queries

```
SELECT pg_size_pretty(pg_database_size(current_database()));
SELECT pid, state, now() - query_start AS running_for, left(query, 60)
FROM pg_stat_activity WHERE state <> 'idle' ORDER BY running_for DESC;
SELECT pid, pg_blocking_pids(pid) FROM pg_stat_activity
WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

Indexes and plans

```
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 42;
CREATE INDEX CONCURRENTLY idx_orders_customer ON orders (customer_id);
ANALYZE orders;
```

In the plan	Meaning
Seq Scan on a big table	Reads every row, often needs an index
Estimate far from actual	Stale statistics, run `ANALYZE`
Sort spilling to disk	Needs a better index or more `work_mem`

Timeouts

```
ALTER ROLE app SET statement_timeout = '30s';
ALTER ROLE app SET lock_timeout = '5s';
ALTER ROLE app SET idle_in_transaction_session_timeout = '30s';
SELECT pg_cancel_backend(1234);
```

Backups

```
pg_dump --format=custom --file=shop.dump shop
pg_restore --dbname=shop_check --jobs=4 shop.dump
pg_basebackup -h db1.example.com -D /backups/base -X stream -P
```

A backup counts only after a successful restore test. A replica is not a backup.

Replication

```
SELECT client_addr, state, pg_wal_lsn_diff(sent_lsn, replay_lsn) AS bytes_behind
FROM pg_stat_replication;
SELECT slot_name, active FROM pg_replication_slots;
```

Failover needs quorum, fencing, promotion and client redirection.

MARGA by EknathaLabs marga.eknathalabs.com

Safe migrations

Change	Safer approach
New index	`CREATE INDEX CONCURRENTLY`
NOT NULL	`CHECK ... NOT VALID`, then `VALIDATE CONSTRAINT`
Rename a column	Expand and contract over several deploys
Large backfill	Small batches, watch replica lag
Any ALTER	Set `lock_timeout` first

Redis quick checks

```
redis-cli INFO memory | grep used_memory_human
redis-cli INFO stats | grep -E "keyspace_hits|keyspace_misses"
redis-cli --scan --pattern 'session:*' | head
```

Use `SCAN`, never `KEYS *`, on production. Set `maxmemory` and an eviction policy.

Danger: `DELETE` *or* `UPDATE` *without* `WHERE` *changes every row. Use a transaction and check the row count before you commit.*

Warning: *A plain* `CREATE INDEX` *blocks writes while it builds. Use* `CONCURRENTLY` *on busy tables.*

Danger: *Never delete files inside the data directory or* `pg_wal` *by hand. It can destroy the database.*

Warning: `KEYS *` *on a production Redis blocks every client. Use* `SCAN`.

MARGA by EknathaLabs
marga.eknathalabs.com